

17. Design of max pooling, min pooling and average pooling layer using RTL

(a) Design of max pooling layer using Vivado

AIM

To design and simulate a Max Pooling layer using Verilog HDL to select the maximum pixel value from a 2×2 input window.

SOFTWARE REQUIRED

- Xilinx Vivado Design Suite
- Vivado Simulator (XSIM)
- Verilog HDL
- Windows / Linux Operating System

PROCEDURE

1. Open Xilinx Vivado and create a new RTL project.
2. Select the target FPGA device or board.
3. Create a Verilog module for the Max Pooling layer.
4. Apply a 2×2 pooling window with 8-bit pixel inputs.
5. Use if-else logic to compare input pixels and determine the maximum value.
6. Write a testbench to apply different pixel values.
7. Simulate the design using Vivado Simulator.
8. Verify the output waveform for correct maximum value selection.

Verilog program for Max-pooling

```
module max_pool_2x2 (  
    input wire [7:0] p0, p1, p2, p3,  
    output reg [7:0] max_out  
);  
  
always @(*) begin  
    if (p0 >= p1 && p0 >= p2 && p0 >= p3)  
        max_out = p0;  
    else if (p1 >= p0 && p1 >= p2 && p1 >= p3)  
        max_out = p1;
```

```

    else if (p2 >= p0 && p2 >= p1 && p2 >= p3)
        max_out = p2;
    else
        max_out = p3;
end
endmodule

```

Testbench program for Max-pooling

```

`timescale 1ns / 1ps

module tb_max_pool_2x2;

    reg [7:0] p0, p1, p2, p3;
    wire [7:0] max_out;

    // Instantiate DUT
    max_pool_2x2 uut (
        .p0(p0),
        .p1(p1),
        .p2(p2),
        .p3(p3),
        .max_out(max_out)
    );

    initial begin
        // Test case 1
        p0 = 8'd100; p1 = 8'd150; p2 = 8'd90; p3 = 8'd200;
        #10;

        // Test case 2
        p0 = 8'd20; p1 = 8'd40; p2 = 8'd60; p3 = 8'd80;
        #10;

        // Test case 3
        p0 = 8'd255; p1 = 8'd0; p2 = 8'd128; p3 = 8'd64;
        #10;

        // Test case 4
        p0 = 8'd10; p1 = 8'd10; p2 = 8'd10; p3 = 8'd10;
    end
endmodule

```

```

        #10;

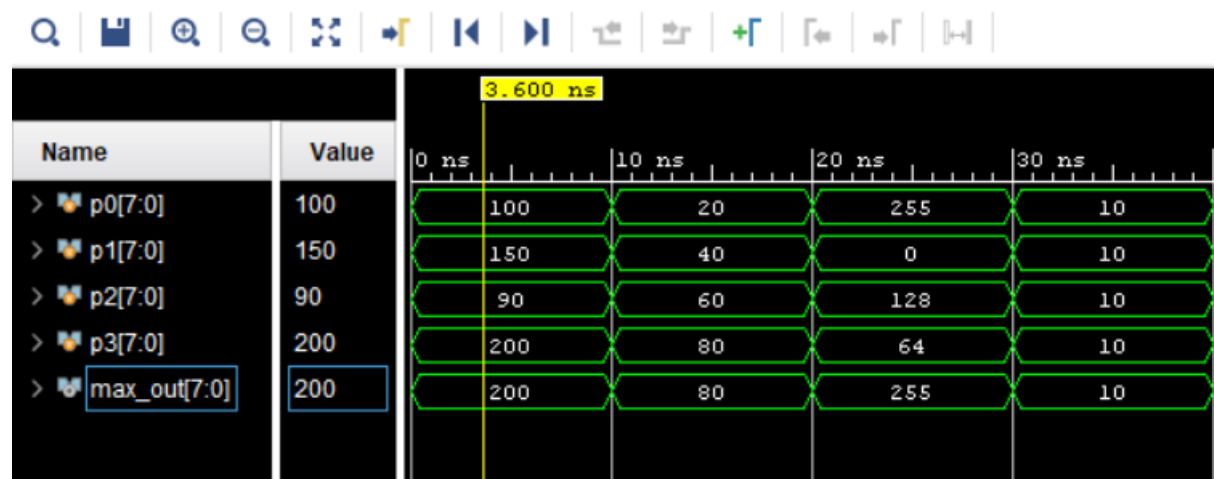
        $stop;

    end

endmodule

```

Output



RESULT

The Max Pooling layer was successfully designed and simulated. For each 2×2 input window, the module correctly produced the maximum pixel value. The simulation results matched the expected values, confirming correct operation of the Max Pooling layer.

(b) Design of min pooling layer using Vivado

AIM

To design and simulate a Min Pooling layer using Verilog HDL to select the minimum pixel value from a 2×2 input window.

SOFTWARE REQUIRED

- Xilinx Vivado Design Suite
- Vivado Simulator (XSIM)
- Verilog HDL
- Windows / Linux Operating System

PROCEDURE

1. Create a new RTL project in Xilinx Vivado.
2. Select the target FPGA device or board.
3. Develop a Verilog module for the Min Pooling layer.

4. Apply a 2×2 pooling window with 8-bit pixel inputs.
5. Use if–else logic to compare input pixels and determine the minimum value.
6. Write a testbench to apply different input combinations.
7. Simulate the design using Vivado Simulator.
8. Observe the output waveform and verify minimum value selection.

Verilog program for Min-pooling

```
module min_pool_2x2 (  
    input wire [7:0] p0, p1, p2, p3,  
    output reg [7:0] min_out  
);  
always @(*) begin  
    if (p0 <= p1 && p0 <= p2 && p0 <= p3)  
        min_out = p0;  
    else if (p1 <= p0 && p1 <= p2 && p1 <= p3)  
        min_out = p1;  
    else if (p2 <= p0 && p2 <= p1 && p2 <= p3)  
        min_out = p2;  
    else  
        min_out = p3;  
end  
endmodule
```

Testbench program for Min-pooling

```
`timescale 1ns / 1ps  
module tb_min_pool_2x2;  
    reg [7:0] p0, p1, p2, p3;  
    wire [7:0] min_out;  
    // Instantiate DUT  
    min_pool_2x2 uut (  
        .p0(p0),  
        .p1(p1),
```

```

.p2(p2),
.p3(p3),
.min_out(min_out)
);

initial begin
    // Test case 1
    p0 = 8'd100; p1 = 8'd150; p2 = 8'd90; p3 = 8'd200;
    #10;

    // Test case 2
    p0 = 8'd20; p1 = 8'd40; p2 = 8'd60; p3 = 8'd80;
    #10;

    // Test case 3
    p0 = 8'd255; p1 = 8'd0; p2 = 8'd128; p3 = 8'd64;
    #10;

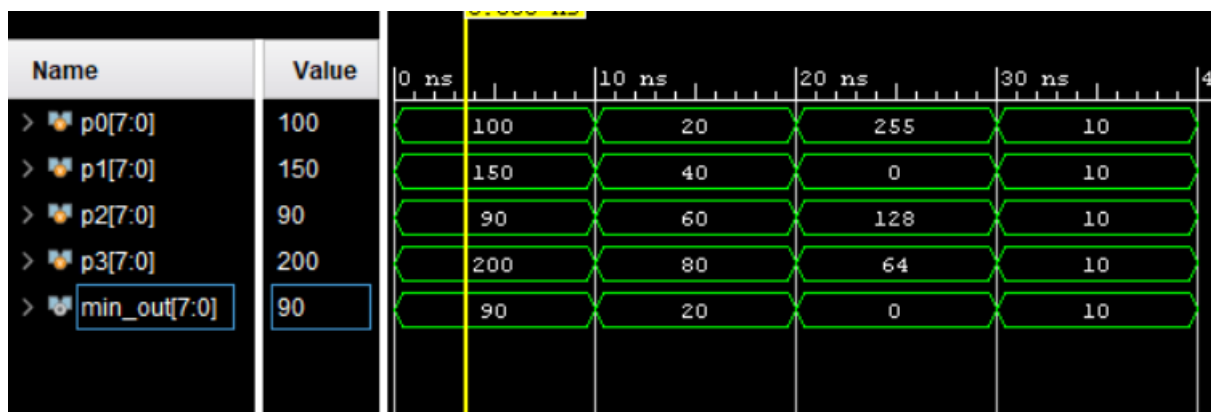
    // Test case 4
    p0 = 8'd10; p1 = 8'd10; p2 = 8'd10; p3 = 8'd10;
    #10;

    $stop;
end

endmodule

```

Output



RESULT

The Min Pooling layer was successfully designed and simulated.
For each 2×2 input window, the module correctly generated the minimum pixel value.
The simulation results were verified and found to be correct.

(c) Design of average pooling layer using Vivado

AIM

To design and simulate an Average Pooling layer using Verilog HDL to compute the average of pixel values in a 2×2 input window.

SOFTWARE REQUIRED

- Xilinx Vivado Design Suite
- Vivado Simulator (XSIM)
- Verilog HDL
- Windows / Linux Operating System

PROCEDURE

1. Launch Xilinx Vivado and create a new RTL project.
2. Select the target FPGA device or board.
3. Create a Verilog module for the Average Pooling layer.
4. Apply four 8-bit pixel inputs corresponding to a 2×2 window.
5. Compute the sum of the input pixels and divide by four to obtain the average.
6. Develop a testbench to apply sample pixel values.
7. Simulate the design using Vivado Simulator.
8. Verify the average value from the output waveform.

Verilog program for average-pooling

```
module avg_pool_2x2 (  
    input wire [7:0] p0, p1, p2, p3,  
    output reg [7:0] avg_out  
);  
  
wire [9:0] sum; // Max sum = 4 × 255 = 1020  
assign sum = p0 + p1 + p2 + p3;  
always @(*) begin  
    avg_out = sum >> 2; // Divide by 4
```

```
end
```

```
endmodule
```

Testbench program for average-pooling

```
`timescale 1ns / 1ps
```

```
module tb_avg_pool_2x2;
```

```
    reg [7:0] p0, p1, p2, p3;
```

```
    wire [7:0] avg_out;
```

```
    // Instantiate DUT
```

```
    avg_pool_2x2 uut (
```

```
        .p0(p0),
```

```
        .p1(p1),
```

```
        .p2(p2),
```

```
        .p3(p3),
```

```
        .avg_out(avg_out)
```

```
    );
```

```
    initial begin
```

```
        // Test case 1
```

```
        p0 = 8'd100; p1 = 8'd150; p2 = 8'd90; p3 = 8'd200;
```

```
        #10;
```

```
        // Test case 2
```

```
        p0 = 8'd20; p1 = 8'd40; p2 = 8'd60; p3 = 8'd80;
```

```
        #10;
```

```
        // Test case 3
```

```
        p0 = 8'd255; p1 = 8'd0; p2 = 8'd128; p3 = 8'd64;
```

```
        #10;
```

```
        // Test case 4
```

```
        p0 = 8'd10; p1 = 8'd10; p2 = 8'd10; p3 = 8'd10;
```

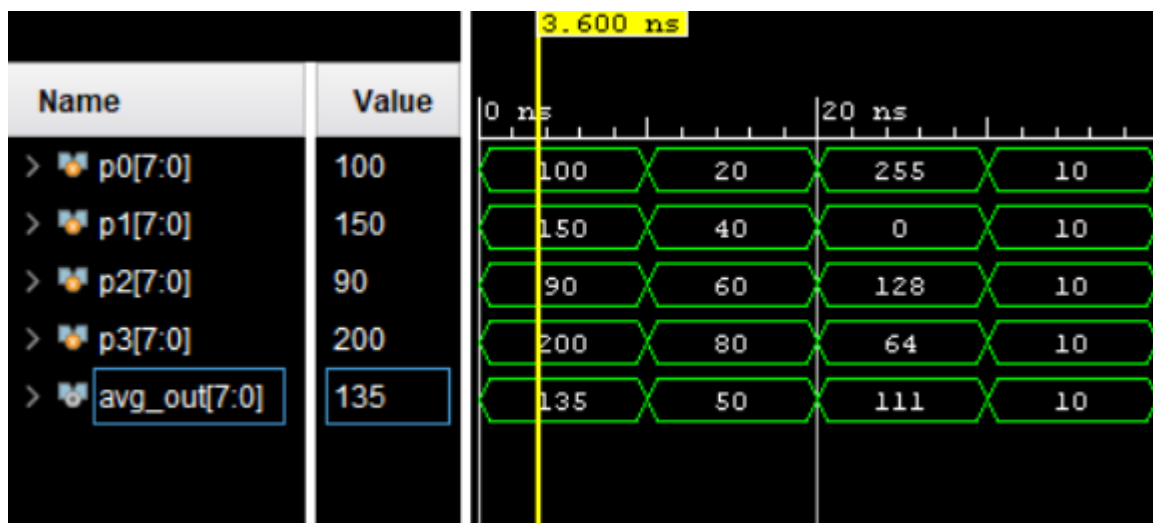
```
        #10;
```

```
        $stop;
```

```
    end
```

```
endmodule
```

Output



RESULT

The Average Pooling layer was successfully designed and simulated. The module correctly computed the average pixel value for each 2x2 input window. Simulation results matched the expected theoretical values, confirming correct functionality.